

EDUCOM

E-COMMERCE PLATFORM

PROJECT DOCUMENTATION REPORT

Stack	MongoDB · Express.js · React · Node.js
Version	1.0.0 — Production
Frontend	educomecommerce.vercel.app
Backend	educom-e-commerce-platform.onrender.com
Month / Year	March 2026

Confidential — Internal Documentation

TABLE OF CONTENTS

Contents

1	Project Overview	3
2	MERN Architecture	3
3	Database Schema	4
4	Authentication & Security	5
5	Redux State Management	6
6	Stripe Payment Integration	7
7	API Reference	7
8	Email Notification System	8
9	Image Upload — Cloudinary	9
10	E-Commerce Best Practices	9
11	Deployment & DevOps	10
12	Conclusion	10

1 Project Overview

EduCom is a full-stack production-grade e-commerce platform built on the MERN stack. It supports the complete retail lifecycle — product discovery, wishlist management, cart operations, Stripe-powered checkout, order tracking, and post-purchase email notifications. The frontend is deployed on Vercel, backend on Render, with media assets on Cloudinary.

- **Authentication** — JWT + bcryptjs, role-based access (user/admin)
- **Cart & Wishlist** — Server-side cart with price snapshotting and coupon support
- **Stripe Payments** — Hosted Checkout with webhook verification and fallback polling
- **Email Notifications** — Resend HTTP API — login alerts, order confirmations, status updates
- **Cloudinary Images** — Drag-and-drop multi-image upload with cover selection
- **Coupon System** — Admin discount codes — percentage/fixed, usage caps, expiry dates

2 System Architecture — MERN Stack

The application follows a three-tier architecture: React SPA on the presentation tier, Express.js REST API on the application tier, and MongoDB as the persistence tier. Layers communicate exclusively via JSON over HTTP, enabling independent scaling and deployment.

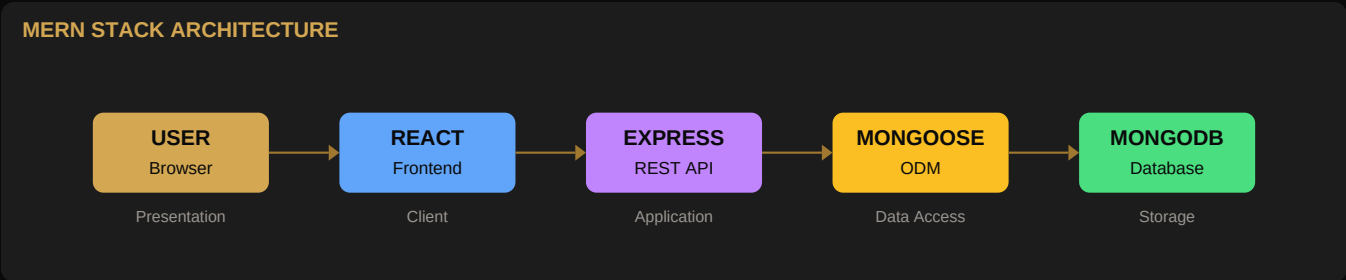


Figure 1 — MERN stack data flow from browser to database

Layer	Directory	Responsibility
Models	backend/models/	Mongoose schemas — User, Product, Order, Cart, Coupon
Controllers	backend/controllers/	Business logic — auth, orders, cart, wishlist, coupons
Routes	backend/routes/	Express routers mapping HTTP verbs to controllers
Middleware	backend/middleware/	Auth guard, admin guard, multer upload, error handler
Services	backend/services/	Stripe session creation and webhook processing
Utils	backend/utils/	JWT generator, email sender, Cloudinary wrapper, templates
Validators	backend/validators/	express-validator rules for input sanitisation

3 Database Schema Design

MongoDB is used as a document store with Mongoose providing schema validation, virtuals, and pre-save hooks. The design embeds frequently co-read data (reviews inside Product) and references independently queried collections (User ↔ Order). All ObjectId references include populated paths in API responses.

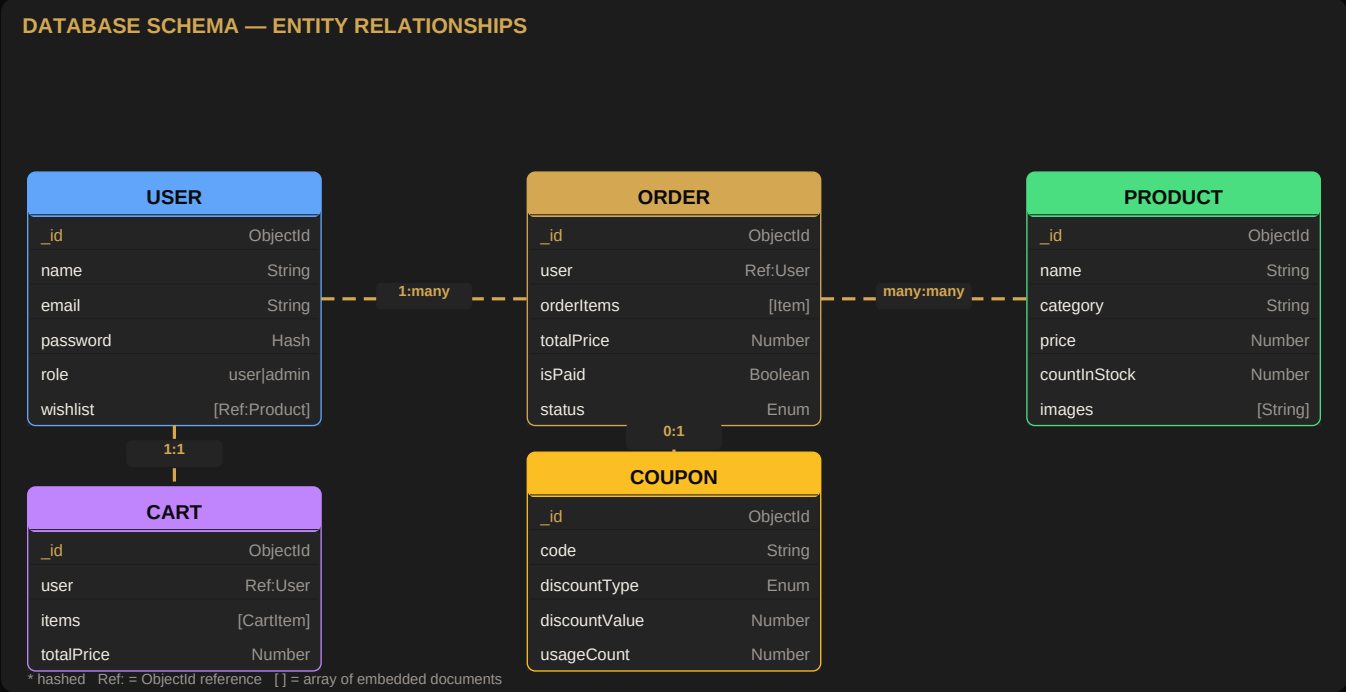


Figure 2 — Entity relationship diagram showing collection references

Key Design Decisions

- **Cart as a separate collection** — One document per user. A pre-save hook recalculates totalPrice on every write, avoiding stale totals.
- **Price snapshotting** — CartItem stores the price at time of adding. Price changes do not retroactively affect existing carts.
- **Reviews embedded in Product** — Always read alongside the product, making embedding more efficient than a separate collection join.
- **Coupon server-side re-validation** — usageCount and maxUsage are re-checked on every order creation. The frontend preview is never trusted alone.
- **Soft-delete via isActive** — Users are deactivated rather than deleted to preserve order history referential integrity.

4 Authentication & Security

Authentication uses JSON Web Tokens with bcryptjs password hashing. The system is stateless — no sessions or cookies are stored server-side. Role-based access control separates user and admin capabilities via chainable middleware.

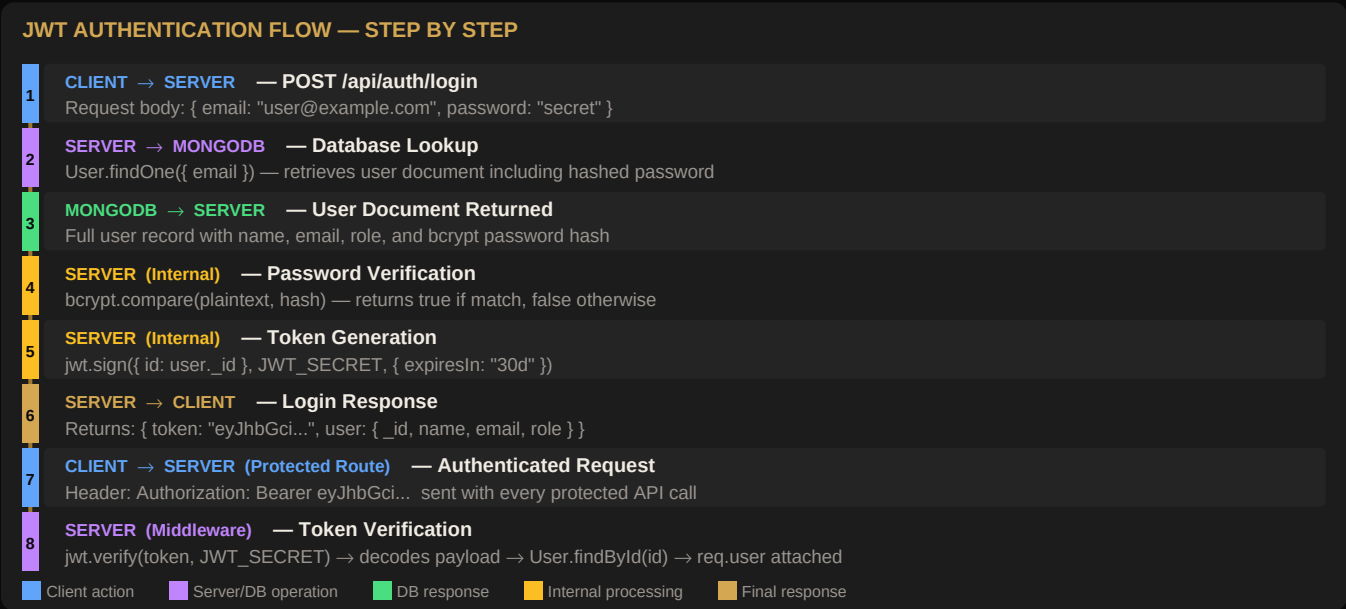


Figure 3 — JWT authentication sequence diagram

Concern	Implementation	Location
Password storage	bcryptjs salt rounds=10	User.js pre-save hook
Token signing	JWT HS256, 30-day expiry	utils/generateToken.js
Token verification	Bearer header + jwt.verify()	middleware/authMiddleware.js
Role enforcement	adminOnly chains after protect	middleware/adminMiddleware.js
Input validation	express-validator on auth endpoints	validators/authValidator.js
CORS restriction	Whitelist single FRONT_END_URL	app.js
Stripe webhooks	constructEvent() signature check	services/stripeService.js

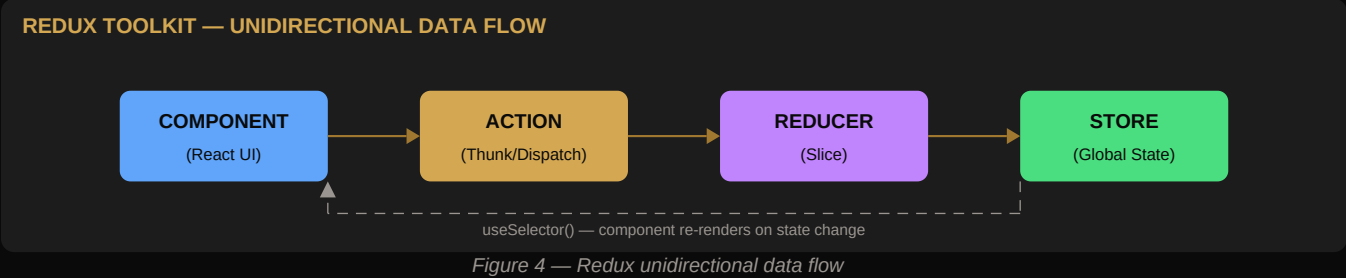
Lazy Initialisation Pattern

Because ES module imports execute before dotenv.config() runs, all third-party SDK clients (Stripe, Cloudinary, Resend) use a getter function that initialises on first use.

```
let _stripe = null;
const getStripe = () => {
  if (!_stripe) _stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
  return _stripe;
};
```

5 Redux State Management

Global state is managed with Redux Toolkit. `createAsyncThunk` handles async API calls with automatic pending/fulfilled/rejected lifecycle actions. `redux-persist` persists the auth and cart slices to `localStorage` so users stay logged in and retain their cart across refreshes.

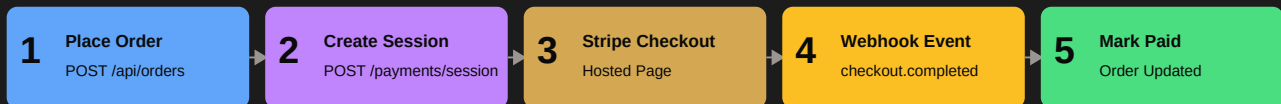


Slice	Persisted	Key State	Async Thunks
auth	Yes	user, token, loading, error	loginUser, registerUser, fetchProfile
products	No	items, selectedProduct, filters	fetchProducts, fetchProductById, createProduct
cart	Yes	items, totalPrice, coupon	fetchCart, addToCart, updateCartItem, removeFromCart
wishlist	No	items, loading	fetchWishlist, toggleWishlist, removeFromWishlist
orders	No	myOrders, allOrders, analytics	createOrder, fetchMyOrders, updateOrderStatus

6 Payment Integration — Stripe

Payments use Stripe Checkout — a hosted page that handles PCI compliance, card validation, and 3D Secure automatically. The backend creates a Checkout Session and returns a redirect URL. On success Stripe sends a webhook event to mark the order paid.

STRIPE PAYMENT INTEGRATION FLOW



Webhook verified with `STRIPE_WEBHOOK_SECRET` · Frontend polls `/api/payments/verify` as local dev fallback

Figure 5 — End-to-end Stripe payment flow

Local Development Fallback

Stripe webhooks cannot reach localhost. `PaymentSuccessPage` polls `GET /api/payments/verify/:orderId` up to 5 times at 2-second intervals. If still unpaid after all polls it calls `PATCH /api/orders/:id/pay` directly. In production the webhook fires immediately and the first poll returns `isPaid: true`.

7 API Reference

Auth & Users

POST	/api/auth/register	Public	Register new user, returns JWT
POST	/api/auth/login	Public	Login, returns JWT and sends notification email
GET	/api/users/profile	Auth	Get own profile with populated wishlist
PUT	/api/users/profile	Auth	Update name, email or password
GET	/api/admin/users	Admin	List all registered users

Products

GET	/api/products	Public	List with keyword/category/sort/page filters
POST	/api/products	Admin	Create product with Cloudinary image URLs
GET	/api/products/:id	Public	Single product with reviews
PATCH	/api/products/:id	Admin	Update product fields
DELETE	/api/products/:id	Admin	Delete product
POST	/api/products/:id/reviews	Auth	Add a product review

Cart & Wishlist

GET	/api/cart	Auth	Get current user cart
POST	/api/cart/add	Auth	Add item with stock validation
PATCH	/api/cart/update	Auth	Update item quantity
DELETE	/api/cart/remove/:id	Auth	Remove single item
DELETE	/api/cart/clear	Auth	Empty entire cart
GET	/api/wishlist	Auth	Get saved products
POST	/api/wishlist/toggle	Auth	Add/remove from wishlist

Orders & Payments

POST	/api/orders	Auth	Create order — applies coupon, deducts stock, clears cart
GET	/api/orders/my	Auth	Get own order history
GET	/api/orders/:id	Auth	Single order (owner or admin)
PATCH	/api/orders/:id/pay	Auth	Mark paid — fallback for no webhook
GET	/api/orders/admin/orders	Admin	All orders with revenue total
PATCH	/api/orders/admin/orders/:id/status	Admin	Update fulfillment status — triggers email
GET	/api/orders/admin/analytics	Admin	Revenue totals, monthly chart, top products
POST	/api/payments/create-checkout-session	Auth	Create Stripe session URL

GET	/api/payments/verify/:id	Auth	Poll payment status
POST	/api/payments/webhook	Stripe	Mark order paid via webhook event

Coupons & Uploads

POST	/api/coupons/validate	Auth	Validate code and preview discount amount
GET	/api/coupons	Admin	List all coupons
POST	/api/coupons	Admin	Create coupon
PUT	/api/coupons/:id	Admin	Update or toggle active status
DELETE	/api/coupons/:id	Admin	Delete coupon
POST	/api/upload/multiple	Admin	Upload images to Cloudinary
DELETE	/api/upload	Admin	Delete image from Cloudinary by publicId

8 Email Notification System

Transactional emails are delivered via Resend HTTP API, chosen over SMTP because Render's free tier blocks all outbound SMTP ports (25, 465, 587). Resend uses HTTP which is never firewall-restricted. All emails are HTML-formatted with the EduCom brand theme.

Trigger	Subject	Content
User login	New sign-in to your account	Timestamp (IST), user-agent string, security advice
Order created	Order Confirmed — #ID	Full itemised receipt with prices and shipping address
Status: processing	Your order is being processed	Payment confirmed, items being prepared
Status: shipped	Your order is on its way!	Shipping confirmation with estimated delivery note
Status: delivered	Your order has been delivered	Delivery confirmation message
Status: cancelled	Your order has been cancelled	Cancellation notice with refund timeline

- **Fire-and-forget with full logging** — Email errors are logged to Render's log stream but never propagate to the API response.
- **Centralised templates** — All HTML lives in emailTemplates.js with a shared layout() wrapper. Rebranding requires editing one file.
- **Resend free tier** — 3,000 emails/month. From address uses onboarding@resend.dev until a custom domain is verified.

9 Image Upload — Cloudinary

Product images are stored on Cloudinary CDN. The backend uses multer memoryStorage (no disk writes) to receive multipart uploads, then streams each buffer directly to Cloudinary via upload_stream. Returned secure_url values are stored in Product.images[].

Step	Where	Action
1	Frontend	Admin drops files onto ImageUploader component
2	Frontend → Backend	POST /api/upload/multiple (multipart/form-data)
3	Backend	multer memoryStorage — file arrives as Buffer, no disk I/O
4	Backend → Cloudinary	uploadToCloudinary() streams buffer via upload_stream
5	Cloudinary	Stores image, applies quality:auto and fetch_format:auto (WebP)
6	Backend → Frontend	Returns { url, publicId } array
7	Frontend	URLs stored in product form images[] array
8	Frontend → Backend	POST /api/products includes images[] of Cloudinary URLs

10 E-Commerce Best Practices

Security

- Passwords hashed with bcryptjs (salt=10) — plain-text passwords never stored or logged
- JWT tokens signed with long random secret, expire after 30 days
- All admin routes protected by protect + adminOnly middleware chains
- Stripe webhook signature verification prevents forged payment confirmations
- Environment variables loaded via dotenv, never committed to version control

Performance

- MongoDB text index on Product name + description for fast keyword search
- Pagination on all product list endpoints (configurable limit, default 12)
- React lazy() + Suspense for all page components — reduces initial bundle size
- Cloudinary auto-format serves WebP to modern browsers automatically
- redux-persist whitelist avoids persisting large product lists to localStorage

UX & Reliability

- Skeleton loaders prevent layout shift during data fetching
- Array.isArray() guards on all Redux selectors prevent crashes during rehydration
- Cart price snapshotting — adding to cart locks the price at that moment
- Stock decremented at order creation, not at payment — prevents overselling
- Stripe polling fallback handles webhook delivery failures in development

11 Deployment & DevOps

Service	Provider	Notes
Frontend	Vercel	Auto-deploy on git push. vercel.json rewrites for React Router SPA routing
Backend	Render Free	Spins down after 15min inactivity. All env vars set in Render dashboard
Database	MongoDB Atlas M0	Free cluster. IP whitelist 0.0.0.0/0 for Render compatibility
Media CDN	Cloudinary	Free tier. Auto WebP. Folder: educom/products
Email	Resend	HTTP API. 3000 emails/month free. No SMTP port restrictions on Render
Payments	Stripe Test	Hosted checkout. Webhook registered in Stripe dashboard for production

Variable	Service	Purpose
MONGO_URI	Atlas	Connection string with credentials
JWT_SECRET	App	Long random string for signing tokens
FRONT_END_URL	App	Vercel domain for CORS and Stripe redirect URLs
STRIPE_SECRET_KEY	Stripe	sk_test_... or sk_live_... from Stripe dashboard
STRIPE_WEBHOOK_SECRET	Stripe	whsec_... from Stripe webhook endpoint settings

Variable	Service	Purpose
CLOUDINARY_CLOUD_NAME	Cloudinary	Cloud name from Cloudinary console
CLOUDINARY_API_KEY	Cloudinary	API key from Cloudinary console
CLOUDINARY_API_SECRET	Cloudinary	API secret from Cloudinary console
RESEND_API_KEY	Resend	re_... from Resend API keys page
EMAIL_FROM_NAME	Email	Sender display name shown in inbox
EMAIL_FROM_ADDRESS	Email	onboarding@resend.dev or verified custom domain

12 Conclusion

EduCom is a production-ready full-stack e-commerce platform demonstrating modern web development best practices across every layer of the stack. The MERN architecture provides clean separation of concerns, Redux Toolkit delivers predictable client state, and Stripe, Cloudinary, and Resend bring real-world commercial capabilities to the platform.

The codebase is structured for scalability — new features can be added as independent controller/route/slice units without modifying existing code. The lazy initialisation pattern, server-side re-validation of all business rules, and comprehensive error handling make the system robust against both developer mistakes and malicious input.

Future Enhancement Roadmap

- **Custom domain email** — Verify a domain in Resend to send from noreply@educom.com
- **Stripe webhooks in production** — Register the webhook endpoint in Stripe dashboard for reliable payment confirmation
- **Search enhancement** — Replace MongoDB text search with Algolia for typo-tolerance
- **Rate limiting** — Add express-rate-limit on auth endpoints to prevent brute-force
- **Refresh tokens** — Short-lived access tokens (15 min) with long-lived refresh tokens
- **PDF invoices** — Generate and attach PDF invoices to order confirmation emails

EduCom E-Commerce Platform — Full-Stack MERN Application

Documented March 2026 · MongoDB · Express.js · React · Node.js